Algoritmi Aproximativi - Temă Seminar

Bercaru Alexandru-Mihai, grupa 231

Aprilie 2025

1 Knapsack

a) Fie următorul algoritm scris în pseudo-cod (mai jos regăsim și implementarea în limbajul Python):

Algorithm 1 (MaxSum(S,K))

Inițializez matricea $M \in \mathcal{M}_{(n+1,K+1)}(\mathbb{N})$ cu 0
for $i \leftarrow 1, 2, \ldots, n$ **do**
  for $k \leftarrow 1, 2, \ldots, K$ **do**
    if $k < s_i$ **then**
      $M(i,k) = M(i-1,k)$
    else
      $M(i,k) = \max(M(i-1,k), M(i-1,k-s_i) + s_i)$
    end if
  end for
end for
$OPT \leftarrow M(n,K)$

```python
def alg1(S, K):
      dp = [[0] * (K+1) for i in range(len(S))]
      for idx in range(1, len(S)):
          for k in range(1, K+1):
              if S[idx] > k:
                  dp[idx][k] = dp[idx-1][k]
              else:
                  dp[idx][k] = max(dp[idx-1][k], dp[idx-1][k - S[idx]] + S[idx])
      return dp[len(S)-1][K]

file = open(path, 'r')
nrstream, K = file.read().split("\n")
numere =[0] + [int(x) for x in nrstream.split()]
rez1 = alg1(numere, int(K))
print(f'OPT={rez1}')
```

Explicație: La fiecare pas, calculez dacă aș obține o sumă mai mare adăugând numărul s_i la suma deja obținută, știind că suma totală trebuie să fie $\leq k$, unde k reprezintă suma parțială maximă pe care o pot avea la acest pas (k va lua toate valorile de la 1 la K, fiind coloana pe care mă aflu la un moment al execuției). Voi alege maximul dintre sum_1 și $sum_2 + s_i$, unde:
sum_1 := suma maximă care se poate obține din primele $i-1$ numere, care să fie $\leq k$
sum_2 := suma maximă care se poate obține din primele $i-1$ numere, care să fie $\leq k - s_i$.

Complexitatea spațiu: $\mathcal{O}(n * K)$, deoarece folosesc o matrice cu $n+1$ linii și $K+1$ coloane.
Complexitatea timp: $\mathcal{O}(n * K)$, deoarece parcurg element cu element această matrice.

Demonstrația optimalității (corectitudinii):

Problema se poate împărți în subprobleme de același tip, pe care le compunem pentru a obține soluția (optimul global se determină pe baza optimelor locale).

Subproblema(j, k): aflarea sumei maxime $\leq k$ care poate fi obținută folosind numere din mulțimea $\{s_1, s_2, \ldots, s_j\}$.

Fie R suma maximă $\leq K$ care se poate obține alegând numere din mulțimea $\{s_1, s_2, \ldots, s_{n-1}\}$. Presupunând că am determinat R, a rămas să calculăm suma maximă pe care aș putea să o obțin din $\{s_1, s_2, \ldots, s_{n-1}\} \cup \{s_n\}$. Trebuie să decidem dacă e rentabil sau nu să îl adăugăm pe s_n la soluție, comparând R cu suma dintre s_n și suma maximă obținută dacă am înlătura s_n, i.e. suma maximă pe care o pot obține alegând numere din mulțimea $\{s_1, s_2, \ldots, s_{n-1}\}$, dar care să fie $\leq K - s_n$. Cum $1 \leq s_n \leq K$, ar trebui să cunosc în prealabil sumele maxime care se pot obține alegând dintre primele n-1 numere din șir, în limita valorilor $K-1, K-2, ..., K-K$, adică în limita oricărui număr între 0 și K.

Pentru a calcula suma maximă care se poate obține alegând dintre primele n-1 numere din șir, dar în limita oricărui număr între 0 și K, voi considera că am calculat suma maximă $\leq K$ care se poate obține alegând numere din mulțimea $\{s_1, s_2, \ldots, s_{n-2}\}$ și trebuie să decid dacă este mai rentabil să îl adaug sau nu pe s_{n-1}. Repet acest raționament, ajungând la subprobleme din ce în ce mai mici, până va trebui să calculez suma maximă care se poate obține alegând sau nu pe s_1, în limita oricărui număr între 0 și K.

b) Fie următorul algoritm:

Algorithm 2 (MaxSumApprox(S,K))

$nrmax \leftarrow \max(S)$
$sum \leftarrow 0$
for $i \leftarrow 1, 2, \ldots, n$ **do**
  if $K \geq s_i$ **then**
    $sum \leftarrow sum + s_i$
    $K \leftarrow K - s_i$
  end if
end for
$ALG \leftarrow \max(nrmax, sum)$

```python
def alg2(stream, K):
    suma = 0
    s_max = -1
    for s in [int(x) for x in stream.split()]:
        if s <= K:
            suma += s
            K -= s
        if s > s_max:
            s_max = s
    return max(suma, s_max)

file = open(path, 'r')
nrstream, K = file.read().split("\n")
rez2 = alg2(nrstream, int(K))
print(f'ALG={rez2}')
```

Explicație: Calculez suma, adăugând numerele în ordinea din șir, până când suma lor ar depăși K. La final, rezultatul va fi maximul dintre suma calculată și cel mai mare număr din șir.

Teorema 1. *Algoritmul prezentat mai sus este* $\frac{1}{2}$*-aproximativ.*

Proof. Vom nota cu k **indicele primului număr care nu mai poate fi adăugat la sumă**, cu $nmax$ cel mai mare număr din șir, cu OPT soluția optimă și cu ALG soluția furnizată de algoritmul nostru. Avem:

$$OPT \leq \sum_{1 \leq i \leq k} s_i = \sum_{1 \leq i < k} s_i + s_k \leq \sum_{1 \leq i < k} s_i + nmax$$

Dar:

$$ALG = max(\sum_{1 \le i < k} s_i, nmax) \Rightarrow \sum_{1 \le i < k} s_i \le ALG, nmax \le ALG$$

$$\Rightarrow OPT \le \sum_{1 \le i < k} s_i + nmax \le ALG + ALG = 2ALG$$

$$\Rightarrow ALG \ge \frac{1}{2} OPT$$

□

2 Load Balancing

Problema 2

a) **Afirmația este falsă.** Intuitiv, nu se poate preciza mereu (indiferent de input și de factorii de aproximare) o aceeași ordonare între $ALG_2(I)$ și $2ALG_1(I)$.
Din ipoteză, obținem următoarele inegalități, aplicând definiția:

$$OPT(I) < ALG_1(I) \le 2OPT(I), \forall I \tag{1}$$

$$OPT(I) < ALG_2(I) \le 4OPT(I), \forall I \tag{2}$$

Se poate face o discuție referitoare la factorii de aproximare ai celor doi algoritmi, în funcție de cât de bună este aproximarea pe care o dau:

- dacă factorul de aproximare al lui ALG_2, $\rho_1 = 4$, **este *tight-bounded***, atunci cu certitudine va exista un input I' a.î. $ALG_2(I') = 4OPT(I')$. Înmulțind cu 2 în (1), obținem, $2ALG_1(I') \le 4OPT(I')$, echivalent cu $2ALG_1(I') \le ALG_2(I')$, deci ar exista un set de date de intrare care să satisfacă inegalitatea.
- dacă factorul de aproximare al lui ALG_2, $\rho_1 = 4$, **nu este *tight-bounded***, atunci putem lua un alt factor de aproximare mai bun pentru ALG_2, fie acesta $\rho_2 = 1,99$. Pentru o problemă de minimizare, un algoritm 1,99-aproximativ este și 4-aproximativ, deci nu am contrazice ipoteza!
  Avem următorul șir de inegailtăți:

  $$ALG_2(I) \le 1,99OPT(I) < 2OPT(I) < 2ALG_1(I), \forall I$$

  Se poate observa că, în această situație, indiferent de input, $ALG_2(I) < 2ALG_1(I)$.

Așadar, nu putem preciza cu exactitate, în fiecare caz, că există un set de date de intrare care să satisfacă inegalitatea.

b) **Afirmația este adevărată.** Presupunem prin R.A. că $\exists I$ input a.î. $ALG_1(I) \ge 2ALG_2(I)$. Despre $ALG_2(I)$ știm că este 4-aproximativ, adică $OPT(I) < ALG_2(I) \le 4OPT(I), \forall I$ input. Înmulțind cu 2 inegalitatea, obținem:

$$2OPT(I) < 2ALG_2(I) \le 8OPT(I), \forall I \tag{3}$$

Dar $ALG_1(I)$ este 2-aproximativ, așadar:

$$ALG_1(I) \le 2OPT(I), \forall I \tag{4}$$

Din (3) și (4) obținem:

$$ALG_1(I) \le 2OPT(I) < 2ALG_2(I), \forall I$$

pentru orice I input. Însă, am presupus inițial că există cel puțin un input pentru care $ALG_1(I) \ge 2ALG_2(I)$.
Contradicție!

Problema 3

Teorema 2. *Algoritmul Ordered-Scheduling Algorithm prezentat în curs este* $\frac{3}{2} - \frac{1}{2m}$ *- aproximativ.*

Proof. Vom folosi următoarele notații:

- $cmax :=$ indicele mașinii cu încărcătura maximă în urma rulării algoritmului;
- $q :=$ ultimul job asignat calculatorului $cmax$;
- $load'(c) :=$ încărcătura mașinii c, după ce au fost asignate $q-1$ job-uri (înainte de a asigna job-ul q).

Soluția găsită de algoritm este $ALG = load'(cmax) + t_q$.
Deoarece job-ul q a fost asignat calculatorului $cmax$, putem spune că, la momentul q, încărcătura mașinii $cmax$ era mai mică decât media încărcăturilor la acel moment, adică:

$$load'(cmax) \leq \frac{1}{m}\sum_{i=1}^{m} load'(i) = \frac{1}{m}\sum_{i=1}^{q-1} t_i$$

Evident, suma timpilor alocați activităților până în momentul q va fi mai mică decât suma timpilor tuturor activităților. Să presupunem că excludem activitatea t_q, așadar o scădem din suma totală. Inegalitatea se va menține în continuare:

$$\frac{1}{m}\sum_{i=1}^{q-1} t_i \leq \frac{1}{m}(\sum_{i=1}^{n} t_i - t_q) = \frac{1}{m}\sum_{i=1}^{n} t_i - \frac{1}{m}t_q$$

$$\Rightarrow ALG = load'(cmax) + t_q \leq \frac{1}{m}\sum_{i=1}^{n} t_i - \frac{1}{m}t_q + t_q = \sum_{i=1}^{n} t_i + (1 - \frac{1}{m})t_q$$

Se disting două situații:
Cazul I. Dacă $q \leq m$, atunci activitatea q va fi alocată unei mașini goale la acel moment. $\Rightarrow ALG = t_q \leq t_1 \leq OPT$. Dar $ALG \geq OPT$, deci $ALG = OPT$.
Cazul II. Dacă $q > m$:
Conform Lemei 1 din curs, avem: $\frac{1}{m}\sum_{1\leq i\leq n} t_i$
Deoarece activitățile sunt sortate descrescător după timp, avem: $t_q \leq t_m, t_q \leq t_{m+1} \Rightarrow 2t_q \leq t_m + t_{m+1} \Rightarrow t_q \leq \frac{1}{2}(t_m + t_{m+1})$.
Conform Lemei 3 din curs, $t_m + t_{m+1} \leq OPT$, deci $t_q \leq \frac{1}{2}OPT$. Așadar:

$$ALG \leq \sum_{i=1}^{n} t_i + (1 - \frac{1}{m})t_q \leq OPT + (1 - \frac{1}{m}) * \frac{1}{2}OPT = (\frac{3}{2} - \frac{1}{2m})OPT.$$

□

3 Travelling Salesman Problem

Problema 1

a) Vom folosi următorul rezultat:

Observația 1. *Dacă pentru o problemă nu există un algoritm de decizie în timp polinomial, atunci acea problemă face parte din* $\mathcal{NP} - Hard$.

Presupunem prin reducere la absurd că instanța TSP cu ponderile muchiilor 1 și 2 nu este o problemă $\mathcal{NP} - Hard$. Deci, există un algoritm care verifică faptul că rezultatul obținut este o soluție corectă și care rulează în timp polinomial. **Vom arăta că nu poate exista un astfel de algoritm.**

Fie $G = (V, E)$ un graf simplu, oarecare și construim G', graf complet și ponderat, plecând de la G.

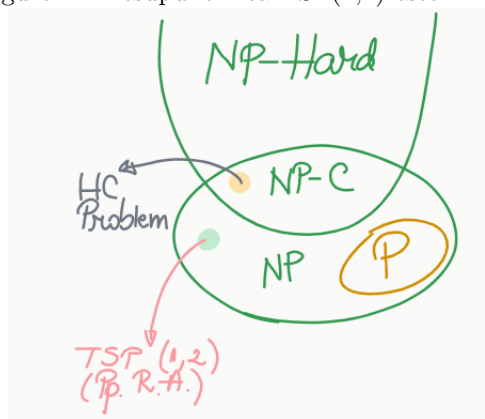
- $G' = (V', E')$
- $V' = V, \mathbf{card}(V') = \mathbf{card}(V) = n$
- $E' = E_1 \cup E_2$, unde $E_1 = E$.
- funcția de cost: $w(e) = \begin{cases} 1, \forall e \in E_1 \\ 2, \forall e \in E_2 \end{cases}$

Practic, am pus costul 1 pe fiecare muchie ce se găsea în G, și costul 2 pe muchiile adăugate pentru a transofrma acel graf într-unul complet. Dacă G are un ciclu hamiltonian, atunci soluția optimă pentru TSP în graful G' va avea costul n, altfel va avea costul minim $n + 1$.
Pentru a decide dacă o soluție la TSP este validă, ar trebui să rezolvăm următoarea problemă de decizie:

Există în G' un ciclu hamiltonian de cost cel mult n?

Dacă pot determina un ciclu hamiltonian, atunci pot afla în timp polinomial dacă costul total este mai mic decât n (parcurg cele n muchii și adun costurile asociate). Însă, aflarea ciclului hamiltonian însuși este o problemă $\mathcal{NP}-Complete$! Deci, problema de decizie este, la rândul ei, $\mathcal{NP}-Complete$. Dar noi am presupus că ar exista un algoritm polinomial de decizie. **Contradicție!**

Figure 1: Presupunem că TSP(1,2) este în NP



b) Fie 3 noduri $u, v, w \in V(G)$ interconectate prin muchiile $e_1, e_2, e_3 \in E(G)$, care respectă $w(e_1) \geq w(e_2) \geq w(e_3)$. Știind că muchiile pot avea ponderi în $\{1, 2\}$, se disting următoarele cazuri:

- dacă $w(e_1) = w(e_2) = w(e_3) = 1 \Rightarrow w(e_2) + w(e_3) = 1 + 1 = 2 \geq 1 = w(e_1)$
- dacă $w(e_1) = 2, w(e_2) = w(e_3) = 1 \Rightarrow w(e_2) + w(e_3) = 1 + 1 = 2 \geq 2 = w(e_1)$
- dacă $w(e_1) = w(e_2) = 2, w(e_3) = 1 \Rightarrow w(e_2) + w(e_3) = 2 + 1 = 3 \geq 2 = w(e_1)$
- dacă $w(e_1) = w(e_2) = w(e_3) = 2 \Rightarrow w(e_2) + w(e_3) = 2 + 2 = 4 \geq 2 = w(e_1)$

Așadar, ponderile satisfac inegalitatea triunghiului.

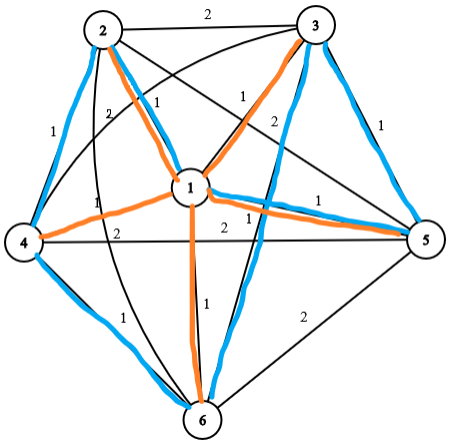
c) Vom arăta că algoritmul din curs, pentru această instanță, **nu** este $\frac{3}{2}$-aproximativ.

Presupunem prin R.A. că în această instanță a problemei, algoritmul este $\frac{3}{2}$-aproximativ. Deci, pentru orice intrare I, avem $ALG(I) \leq \frac{3}{2} OPT(I)$.

Fie graful complet cu 6 noduri, căruia îi asociem următoarele ponderi: $w(e) = \begin{cases} 1, \forall e \in E_1 \\ 2, \forall e \in E_2 \end{cases}$, unde:

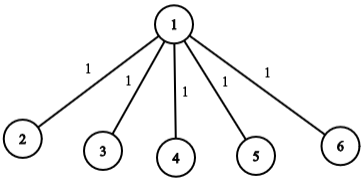
$E_1 = \{(1,2),(1,3),(1,4),(1,5),(1,6),(2,4),(4,6),(3,5),(3,6)\}$
$E_2 = \{(2,3),(3,4),(4,5),(5,6),(2,5),(2,6)\}$.

Figure 2: Graful G descris mai sus. Cu portocaliu sunt desenate muchiile din *apcm*, iar cu albastru muchiile ciclului hamiltonian de cost minim



Dacă, pentru acest graf, vom calcula arborele parțial de cost minim, considerând că muchiile cu același cost le ordonăm lexicografic, atunci *apcm*-ul ar arăta astfel:

Figure 3: Arborele de cost minim



Pentru acest graf, soluția optimă a problemei TSP are costul $OPT = 6$ (putem alege doar muchii cu costul 1, de exemplu: $1 \to 2 \to 4 \to 6 \to 3 \to 5 \to 1$). Algoritmul din curs va parcurge vârfurile în ordinea dată de parcurgerea în adâncime a arborelui parțial, adică: $1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 1$, obținând un cost total egal cu $1+2+2+2+2+1 = 10$. Așadar, $ALG = 10$, $OPT = 6 \Rightarrow \frac{3}{2}OPT = 9$. Am obținut că $ALG > \frac{3}{2}OPT$. **Contradicție!**

4 Vertex Cover

a) Presupunem că avem următoarea expresie cu un număr m de predicate:

$$E = (x_{11} \vee x_{12} \vee x_{13}) \wedge (x_{21} \vee x_{22} \vee x_{13}) \wedge \cdots \wedge (x_{m1} \vee x_{m2} \vee x_{13})$$

Algoritmul prezentat ar putea executa următoarele alegeri:
- prima iterație: alege x_{11} și evaluează la `true`
- a doua iterație: alege x_{21} și evaluează la `true`
. . . . . .
- a m-a iterație: alege x_{m1} și evaluează la `true`.

În final, soluția ar avea dimensiune m. Însă, observăm că, pentru a evalua toate predicatele la `true`, era suficient să asignăm variabilei x_{13} valoarea `true`, întrucât aceasta se regăsește în fiecare predicat. Deci, $OPT(E) = 1$, iar $ALG(E) = m$. Putem aprecia că factorul de aproximare *worst-case* al acestui algoritm este m, depindzând de dimensiunea inputului.

b) Fie următorul algoritm:

Algorithm 3 (3-CNF-Approx)

1: $Sol \leftarrow \emptyset$
2: Fie $C = \{C_1, C_2, \ldots, C_m\}$ mulțimea predicatelor și $X = \{x_1, x_2, \ldots, x_n\}$ mulțimea variabilelor.
3: **while** $C \neq \emptyset$ **do**
4: alegem aleator $C_j \in C$
5: **for** $x_i \in C_j$ **do**
6: $x_i \leftarrow true$
7: adauga(Sol, x_i)
8: elimină din C toate predicatele care conțin variabila x_i
9: **end for**
10: **end while**

Algoritmul este corect, deoarece la fiecare pas se alege un predicat (ceea ce garantează număr finit de pași), iar după eliminare vor rămâne în C doar acele predicate care nu au fost evaluate la `true`, pentru că nu conțineau nicio variabilă dintre cele care au fost evaluate la `true` la pasul curent.

Observația 2. *Fie C^* o mulțime de predicate care nu au variabile comune, i.e. $\bigcap_{P \in C^*} Var(P) = \emptyset$, unde am notat cu $Var(P)$ mulțimea variabilelor care apar în predicatul P. Atunci, $OPT \geq |C^*|$.*

Intuitiv, orice variabilă aș evalua la `true`, îmi va determina cel mult un singur predicat din C^* să se evalueze la `true`, deoarece mulțimile variabilelor sunt disjuncte.

Teorema 3. *Algoritmul 3-CNF-Approx propus mai sus este 3-aproximativ.*

Proof. Mai întâi, observăm că predicatele alese la linia 4 nu conțin variabile comune, deoarece la fiecare pas se elimină toate predicatele care conțin cel puțin una dintre variabilele predicatului selectat. Notăm cu **P** mulțimea predicatelor alese și ale căror variabile au fost evaluate la true. Atunci, conform Observației 2, avem:

$$OPT \geq card(\mathbf{P})$$

$$\Rightarrow 3 * OPT \geq 3 * card(\mathbf{P}) = card(Sol) = ALG$$

□

c) Fie $V = \{v_1, v_2, \ldots, v_n\}$ cu următoarea proprietate: $v_i = \begin{cases} 1, & \text{daca } x_i \in Sol \\ 0, & \text{altfel} \end{cases}$

Scopul este să minimizăm numărul de variabile din soluție, adică $\sum_{1 \leq i \leq n} v_i$, respectând constrângerile:
Dacă x_i, x_j și x_k sunt 3 variabile care fac parte din același predicat, i.e. $\exists C_u \in C$ a.î. $C_u = x_i \vee x_j \vee x_k$, atunci:

$$\begin{cases} v_i + v_j + v_k \geq 1 \\ 0 \leq v_r \leq 1, \forall r \in \{1, 2, \ldots, n\} \end{cases} \tag{5}$$

d) Fie algoritmul:

Algorithm 4 (Polynomial 3-CNF-Approx)

```
Sol ← ∅
Rezolvă următoarea problemă de programare liniară cu numere reale: minimimează ∑_{1≤i≤n} v_i
respectând constrângerile (5).
for i ∈ 1, ..., n do
    if v_i ≥ 1/3 then
        adauga(x_i, Sol)
    end if
end for
```

Teorema 4. *Algoritmul Polynomial 3-CNF-Approx propus mai sus este 3-aproximativ.*

Proof. Fie $OPT_{relaxed}$ soluția găsită de algoritmul de programare liniară cu valori reale (în care $v_i \in [0, 1]$) și OPT soluția optimă pentru $3 - CNF$. Putem aprecia că $OPT_{relaxed} \leq OPT$, deoarece soluția optimă pentru $3 - CNF$ corespunde cu o soluție optimă a versiunii 0/1 pentru problema de programare liniară, iar relaxarea celei de-a doua constrângeri, pentru versiunea cu valori reale, nu poate înrăutăți soluția. Avem:

$$ALG = |Sol| = |\{x_i | v_i \geq \frac{1}{3}\}|$$

$$\Rightarrow ALG = \sum_{x_i \in Sol} 1 \leq \sum_{x_i \in Sol} 3v_i \leq 3 \sum_{i=1}^{n} v_i = 3 * OPT_{relaxed} \leq 3 * OPT$$

□